

SwiftDrop

Real-Time Last-Mile Logistics Platform

Project Report

GitHub: github.com/SMAWAHID/Swiftdrop

Repository	SMAWAHID/Swiftdrop
Stack	FastAPI · PostgreSQL · React · TypeScript
Patterns	Singleton · Factory · Facade (GoF)
Concurrency	SELECT FOR UPDATE NOWAIT (Row-Level Locking)
Auth	JWT + bcrypt
Members	Syed Mawahid · Naheel Siddqui · Muhammad Sohaib
Roll No.	24K-1041 · 24K-0830 · 24K-0577

1. Executive Summary

SwiftDrop is a production-ready, full-stack logistics platform built to solve the concurrency and coordination challenges inherent to last-mile delivery operations. The system connects vendors who book shipments with drivers who fulfill them, using a real-time, role-based interface backed by a rigorously engineered API. The project demonstrates advanced software engineering concepts including Gang-of-Four (GoF) design patterns, database-level concurrency controls, layered architecture, and modern reactive frontend development.

2. Project Objectives

- Design and implement a robust shipment booking and fulfillment workflow
- Apply GoF design patterns (Singleton, Factory, Facade) in a real production context
- Prevent race conditions in concurrent order acceptance using row-level locking • Build role-differentiated dashboards for Vendors, Drivers, and Admins
- Maintain 3NF-normalized PostgreSQL schema with auditing triggers and views

3. Technology Stack

Layer	Technology
Database	PostgreSQL 15+ with PL/pgSQL triggers and views
Backend	Python 3.12, FastAPI, asyncpg
Frontend	React 18, TypeScript, Vite, Axios
Authentication	JWT (python-jose), bcrypt
Architecture	Layered N-Tier (Router → Service → DB)

4. System Architecture

4.1 Backend Architecture

The backend follows a strict layered N-Tier pattern. The **Router Layer** (Controllers) consists of thin HTTP handlers responsible only for parsing requests and serializing responses. They raise domain-specific exceptions rather than HTTP exceptions, keeping business logic out of the transport layer.

The **Service Layer** contains all domain rules. Key services include the shipment service (Facade pattern), accept service (row-level locking), and auth/signup services. The **Data Access Layer** uses raw asyncpg connections with direct SQL control, managed through a Singleton connection pool.

4.2 GoF Design Patterns

Pattern	Location	Purpose
Singleton	db/connection.py	Single shared asyncpg pool across application lifetime
Factory	factories/shipment_factory.py	Produces Standard / Express / Fragile shipment objects
Facade	services/shipment_service.py	Wraps multi-step booking into ShipmentBookingFacade

4.3 Concurrency Control

Problem: Two drivers can simultaneously view a PENDING shipment and both tap 'Accept'. Without locking, both reads see status=PENDING and both writes succeed — two drivers own one shipment.

Solution: `SELECT FOR UPDATE NOWAIT` acquires an exclusive row lock on the shipment record. The first driver's transaction wins the lock. The second driver receives a 409 Conflict immediately — without blocking the HTTP thread.

Why Pessimistic over Optimistic: Only one driver can ever win per shipment. Optimistic locking retries for the loser always fail, wasting compute. Pessimistic locking fails fast with a clean user-facing error.

4.4 Frontend Architecture

The React + TypeScript frontend is structured around: AuthContext (JWT state via useReducer), custom useShipment hook with optimistic UI updates, ProtectedRoute guards for role-based routing, and an Axios repository layer for API communication. Role-specific dashboards (VendorDashboard, DriverDashboard) render different views and actions per authenticated role.

5. Database Design

The PostgreSQL schema is normalized to 3NF. Key tables and features include:

Entity / Feature	Description
users	Base identity: hashed passwords, role enum (VENDOR, DRIVER, ADMIN)
vendor_profiles / driver_profiles	Role-specific data separated from core user table
shipments	Core entity with status enum, timestamps, FK to vendor and driver
reviews	Post-delivery ratings linked to shipment and driver
notifications	Event-driven messages linked to users
PL/pgSQL audit trigger	Reads SET LOCAL app.current_user_id per-request for change auditing
Dashboard view	Pre-aggregated driver performance metrics for Admin dashboard

asynccpg was chosen over SQLAlchemy ORM to maintain direct transaction-level control needed for SET LOCAL session variable injection consumed by the audit trigger.

6. API Reference

Method	Endpoint	Role	Description
POST	/api/auth/login	Any	Authenticate, receive JWT
POST	/api/shipments/book	VENDOR	Book shipment (Facade pattern)
PUT	/api/shipments/{id}/accept	DRIVER	Accept order (row-level lock)
GET	/api/shipments?status=PENDING	Any	List shipments (role-filtered)
GET	/api/drivers/dashboard	ADMIN	Performance metrics view
PATCH	/api/drivers/me/availability	DRIVER	Toggle availability status

Interactive API documentation is auto-generated by FastAPI at /docs (Swagger UI) and /redoc.

7. Security Implementation

JWT Authentication: Tokens issued on login with role claims. Every protected route validates the token and extracts user role before dispatching to the service layer.

Password Hashing: bcrypt via passlib. Passwords are never stored in plaintext.

Role Guards: Enforced at the service layer, not just the router, preventing privilege escalation.

CORS: Configurable via CORS_ORIGINS env variable. Production restricts to known frontend domain.

Validation: Pydantic v2 validates all request bodies at import time. Custom error handler returns user-friendly messages.

8. Setup & Deployment

Database

```
psql -U postgres -c "CREATE USER swiftdrop WITH PASSWORD 'swiftdrop_pass';"
psql -U postgres -c "CREATE DATABASE swiftdrop OWNER swiftdrop;" psql -U
swiftdrop -d swiftdrop -f backend/db/schema.sql psql -U swiftdrop -d
swiftdrop -f backend/db/seed.sql
```

Backend

```
cd backend cp .env.example .env # Configure DATABASE_URL and
SECRET_KEY
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd frontend
npm install
cp .env.example .env.local
npm run dev # Dev server at localhost:5173
npm run build # Production build (zero TypeScript errors)
```

9. Demo Credentials

Users can signup according to their relevant roles and then login.

Role	Email	Password
Admin	admin@swiftdrop.io	Password@123

10. Engineering Decisions Summary

Decision	Rationale
asyncpg over SQLAlchemy	Required for SET LOCAL session variable injection used by the audit trigger
Pydantic v2	model_validate() replaces deprecated from_orm(); validated at import time
Pessimistic locking	Only one driver can win per shipment — optimistic retries always fail for the loser
JWT over sessions	Stateless authentication suits horizontally scalable deployments
FastAPI over Django/Flask	Native async support, auto-generated OpenAPI docs, Pydantic integration

11. Conclusion

SwiftDrop successfully demonstrates that production-grade engineering practices — design patterns, concurrency-safe database operations, layered architecture, and type-safe frontend development — can be cohesively applied in a single, well-scoped project. The platform is deployable as-is and serves as a strong reference implementation for real-time logistics systems.